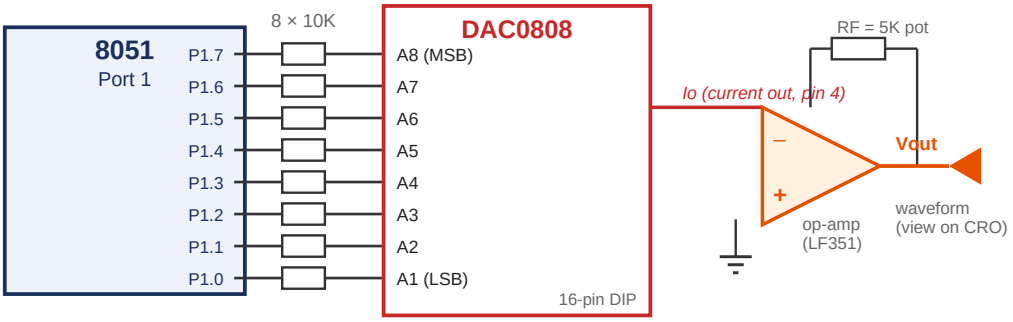
What the exam expects: DAC role • DAC0808 connections • and, for each waveform (square, pulsed, triangular, sawtooth, sine): the **logic**, the **pin pattern on P1** and the **simplified program**. Same circuit generates ALL five waves — only the P1 pattern changes.

1 • Idea — why a DAC makes waveforms

- DAC0808 = **8-bit Digital-to-Analog Converter**: an 8-bit digital input → an **analog output current** (larger number → larger current).
- The op-amp (I–V converter) turns the current into a voltage: **Vout = –Io × RF** — so **Vout ∝ the value written on P1**.
- So a waveform is made by **writing a chosen sequence of numbers to P1 with delays** — the sequence decides the SHAPE, the delay decides the FREQUENCY.

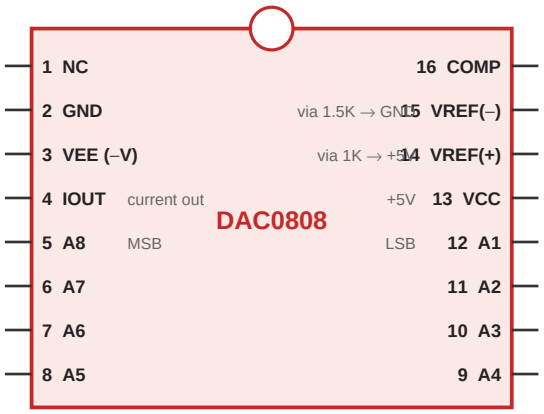


VCC 13 = +5V • VEE 3 = –5V • GND 2 • VREF(+) 14 → 1K → +5V • VREF(–) 15 → 1.5K → GND • COMP 16 → 0.1µF

Digital value on P1 → DAC current Io → op-amp I–V converter → Vout = –Io × RF

SAME circuit for all five waves — only the pattern written on P1 changes

2 • DAC0808 pin diagram (16-pin DIP)

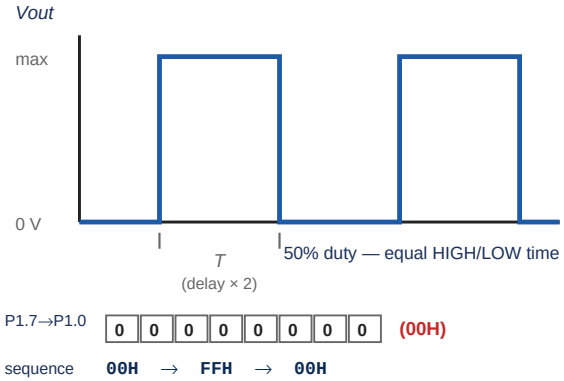


16-pin DIP • numbering:
left 1↓8 top→bottom,
right 9↓16 bottom→top

A1–A8 = 8 digital inputs (LSB→MSB)

Pin roles: A1–A8 = digital inputs (A8 = MSB → connect to P1.7) • VREF(+) sets the reference current • IOUT = analog current out • COMP = compensation capacitor • VCC +5 V, VEE –5 V. Delay subroutine (shared by every program below): `MOV R6,#N / MOV R7,#255 / DJNZ R7 / DJNZ R6 / RET`.

3 • SQUARE WAVE (slide 18 — and slide 13's 5-step procedure)



P1.7→P1.0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | (00H)

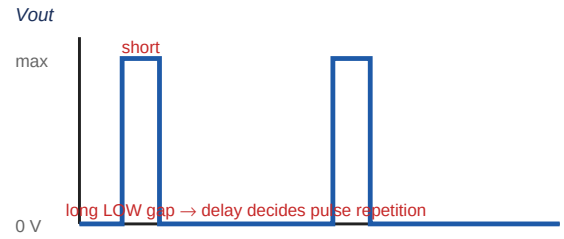
sequence 00H → FFH → 00H

Logic — how it is made:

- Slide's official procedure: **1) Clear P1 2) delay 3) Set P1 4) delay 5) repeat**.
- Only **two levels**: 00H (0 V) and FFH (max V) — the delay between them is the half period.
- **Equal delays** → 50% duty cycle; different delays → square-ish with other duty.

```
MAIN:  MOV P1,#00H     ; LOW level → Vout = 0 V
       ACALL DELAY     ; wait = LOW half-period
       MOV P1,#0FFH    ; HIGH level → Vout = max
       ACALL DELAY     ; equal wait = HIGH half-period
       SJMP MAIN       ; repeat → continuous square wave
```

4 • PULSED WAVE (slide 19)



P1.7→P1.0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | (FFH)

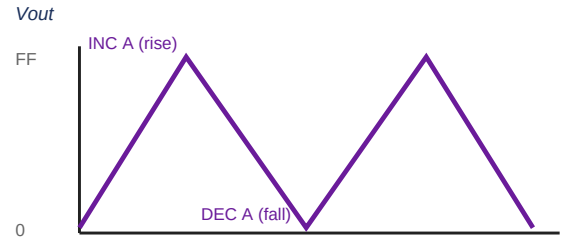
sequence FFH → 00H → FFH

Logic — how it is made:

- Same two levels as square, but **delays are unequal**: very **short HIGH pulse**, **long LOW** gap.
- Pulse width = **short-delay**; time between pulses = short + long delay.
- Use **two separate delay subroutines** with different counts.

```
MAIN:  MOV P1,#0FFH    ; HIGH pulse starts
       ACALL DSHORT    ; SMALL count → narrow pulse
       MOV P1,#00H     ; LOW level
       ACALL DLONG     ; BIG count → long gap
       SJMP MAIN       ; repeat
```

5 • TRIANGULAR WAVE (slide 20)



P1.7→P1.0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | halfway (80H)

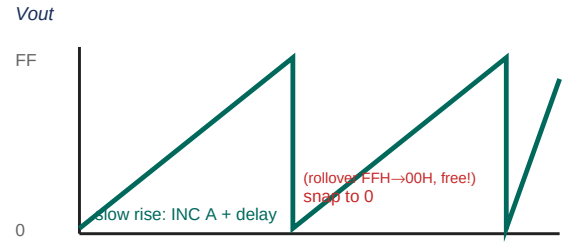
sequence **00H → FFH → 00H**

Logic — how it is made:

- Ramp UP: start **A=00H**, send A to P1, **INC A**, delay, repeat until FFH.
- Ramp DOWN: start **A=FFH**, send A, **DEC A**, delay, repeat (stops before 00H with JNZ).
- Step delay = time of one small step → sets frequency; both ramps equal → symmetric triangle.

```
        MOV A,#00H      ; start at 0 V
UP:     MOV P1,A        ; rising ramp
        ACALL DELAY     ; step time
        INC A
        JNZ UP          ; 00→01→…→FF
        MOV A,#0FFH
DOWN:   MOV P1,A        ; falling ramp
        ACALL DELAY
        DEC A
        JNZ DOWN        ; FF→…→01
        SJMP MAIN       ; repeat
```

6 • SAWTOOTH WAVE (slide 21)



P1.7→P1.0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | quarter (40H)

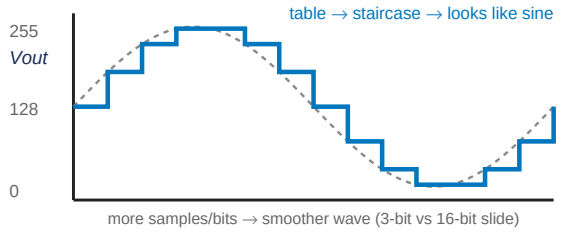
sequence **00H → 7FH → FFH → 00H**

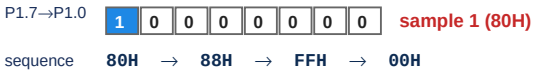
Logic — how it is made:

- **Only a rising ramp**: A = 00H…FFH with delay after each step.
- When A = FFH, **INC A rolls over to 00H automatically** → output snaps to 0 V (vertical fall, no code needed!).
- SJMP UP repeats → teeth; delay sets tooth duration (frequency).

```
        MOV A,#00H      ; tooth starts at 0 V
UP:     MOV P1,A        ; send digital step
        ACALL DELAY     ; controls frequency
        INC A           ; next step
        SJMP UP         ; FFH+1 → 00H rollover!
```

7 • SINE WAVE (slides 16–17 — lookup-table method)





sequence **80H → 88H → FFH → 00H**

Logic — how it is made:

- Sine can't be computed with simple INC/DEC → **pre-compute 100 samples** of one cycle (value = 128 + 127·sin t) and store them in ROM as a **DB table**.
- Program sends the samples one-by-one with **DPTR + MOVC**; after the 100th (DPL = 64H) it restarts.
- Feeding samples quickly gives a **staircase approximation** of sine — more samples/bits → smoother (slide 17).

```
MAIN:  MOV DPTR,#TAB      ; point to table in ROM
NXT:   CLR A
       MOVC A,@A+DPTR     ; read next sample
       MOV P1,A           ; sample → DAC
       ACALL DELAY        ; sample spacing
       INC DPTR
       MOV A,DPL
       CJNE A,#64H,NXT    ; 64H = 100: done?
       SJMP MAIN          ; restart → continuous
TAB:   DB 128,136,144,152 ; 100 samples of
       DB 160,206,…,255   ; 128 + 127·sin t
```

8 • All five waves at a glance (comparison)

Wave	P1 pattern over time	Made by	Frequency set by
Square	`00H ↔ FFH`	Two levels + 2 EQUAL delays	the equal delays
Pulsed	`FFH (short), 00H (long)`	Two UNEQUAL delays	short + long delay
Triangular	`00H→FFH→00H` (ramps)	INC A ramp + DEC A ramp	delay per step
Sawtooth	`00H→FFH, snap 0`	INC A ramp only (rollover)	delay per step
Sine	`100 table samples`	ROM table + MOVC loop	delay + 100 samples

30-second revision: P1 value → DAC current → op-amp → Vout • SHAPE = number sequence, FREQUENCY = delay • square: 00/FF equal • pulsed: unequal • triangle: INC+DEC ramps • sawtooth: INC ramp + rollover • sine: DB table, MOVC, restart after 100 samples.